

FRACMATH: A fast and vectorized MATLAB framework for continuum damage mechanics with crack band regularization

Jaykumar Mavani¹ and Madura Pathirage¹

¹ Gerald May Department of Civil, Construction, and Environmental Engineering, University of New Mexico, Albuquerque, NM, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

FRACMATH is an open-source MATLAB framework for finite-element simulation of crack-band-regularized continuum damage mechanics (CDM) in quasi-brittle materials. The code uses a scalar isotropic damage variable, the modified von Mises equivalent strain (Vree et al., 1995), exponential softening, and Oliver's direction-dependent projected crack-band length (Bažant & Oh, 1983; Oliver, 1989). It is written as a compact MATLAB workflow so users can inspect the model, change the constitutive law, plot results, and debug simulations without a compiled extension layer. Everything stays inside MATLAB: no MEX files, no compiled extensions, and no separate build system.

The package includes a two-dimensional notched three-point bending (3PB) benchmark checked against Abaqus/Standard through an Oliver-matched UMAT, plus two three-dimensional MATLAB benchmarks: the Nooru-Mohamed mixed-mode test and Brokenshire's notched beam torsion test. The repository also stores plotting scripts, benchmark inputs, comparison tables, and the figures used in this paper, so readers can trace the numerical results back to the code paths that produced them. Source code, input decks, results, and a theory manual are available at <https://github.com/Jaykumar9033/FRACMATH> under the MIT license.

Statement of need

Open-source finite-element tools already cover many research needs. Examples include OOFEM in C++ (Patzák & Bittnar, 2001), FEniCS for Python/C++ finite-element workflows (Logg et al., 2012), Akantu for high-performance fracture simulations (Richart et al., 2024), and CALFEM as a MATLAB teaching toolbox (Austrell et al., 2004). These projects are valuable, but using or extending them for a new damage formulation often requires learning a larger architecture, build system, or compiled user-material interface. Commercial workflows such as Abaqus/Standard are also powerful, but custom CDM models generally require UMAT/VUMAT development and careful state-variable handling (Dassault Systèmes Simulia Corp., 2023).

These tools work, but each one often asks the user to learn a new templated C++ interface, Python wrapper, Fortran subroutine, or solver-specific build workflow before changing a single softening curve (Dassault Systèmes Simulia Corp., 2023; Logg et al., 2012; Patzák & Bittnar, 2001). For a graduate student learning damage mechanics, that overhead can become the project.

MATLAB remains attractive in engineering research because it is easy to use: students already know its matrix syntax, plotting tools, debugger, profiler, and sparse linear algebra, and MATLAB has already supported both finite-element teaching software and CDM research codes (Austrell et al., 2004; Eldababy et al., 2025). This matters for fracture modeling, where a user

often needs to inspect element-level strain histories, change a softening law, compare crack-band lengths, and immediately visualize the damage field. FRACMATH uses that accessibility to provide a transparent CDM reference implementation rather than a large general-purpose finite-element platform. It avoids MEX files and external compiled dependencies for the MATLAB solver, which keeps installation simple and makes the implementation suitable for teaching, prototyping, and reviewer-side reproduction.

The closest JOSS work is Parallel-CDM by Eldababy et al. (Eldababy et al., 2025), which provides a MATLAB implementation of 2D local and nonlocal CDM with parallel assembly. The JOSS continuum-damage-mechanics listing currently contains that single CDM-focused paper, making it the most relevant open-source comparison point for this submission. FRACMATH is complementary rather than a replacement. Its contribution is not a new parallel assembly framework. Instead, the novelty is the direction-dependent Oliver crack-band scaling used consistently in MATLAB and Abaqus, the direct UMAT cross-check on an identical 3PB mesh, and the extension of the same scalar damage routine from 2D mode-I fracture to 3D mixed-mode and torsion-driven crack paths. This combination gives users a short MATLAB code base for understanding the regularization procedure and a commercial-code comparison for checking the same constitutive assumptions.

Method

The material model follows standard scalar CDM, where damage variables and equivalent-strain histories are used to represent stiffness degradation in quasi-brittle fracture (Bažant & Planas, 1998; Vree et al., 1995). Damage degrades the undamaged stiffness as

$$\sigma = (1 - \omega) \mathbb{C}_0 : \varepsilon, \quad (1)$$

where $\omega \in [0, 1]$ is the damage variable. A history variable stores the maximum equivalent strain so damage cannot heal. Exponential softening is scaled element by element so the dissipated fracture energy matches G_F after crack-band regularization (Bažant & Oh, 1983). Instead of using a fixed mesh length, FRACMATH computes Oliver's projected characteristic length from the element geometry and the current maximum-principal-strain direction (Oliver, 1989). For a three-node triangle,

$$h(\mathbf{n}) = \frac{2}{\sum_{a=1}^3 |\nabla N_a \cdot \mathbf{n}|}. \quad (2)$$

The same idea is used for tetrahedra in 3D. In the Abaqus comparison, a preprocessing script writes the T3 shape-function gradients to `oliver_t3_gradN.dat`; the UMAT then recomputes Equation 2 from the current principal strain direction. This avoids using Abaqus CELENT as a fixed crack-band length and keeps the regularization consistent with Oliver's characteristic-length construction (Oliver, 1989).

The quasistatic solver uses displacement control and a modified Newton–Raphson loop. At each load step, a secant stiffness is assembled and factored once. Element strains, equivalent strain, history variables, damage, and crack-band lengths are updated with vectorized MATLAB operations, including batched matrix products through `pagetTimes`, while linear systems use MATLAB's sparse backslash interface, which dispatches to UMFPACK (Davis, 2004). The code separates mesh data, material data, state variables, the constitutive update, and postprocessing outputs, but keeps these pieces in plain MATLAB functions. This design is intentional: the routines are short enough for a user to read, yet structured enough to run the same benchmark repeatedly while changing mesh density, softening parameters, or output thresholds.

For the Abaqus comparison, FRACMATH includes the input deck, the Fortran UMAT, the Oliver-gradient table generator, and extraction scripts for load-CMOD and damage fields. The

MATLAB and Abaqus paths therefore share the same mesh, boundary conditions, fracture energy, tensile strength, and crack-band bandwidth formula. Differences in the plotted response are primarily solver and implementation differences, not changes in the continuum model.

Benchmarks

The main benchmark is the Gregoire notched 3PB beam (Grégoire et al., 2013), with $D = 100$ mm, $a/D = 0.2$, span $S = 250$ mm, thickness 50 mm, and a refined CPS3 triangular mesh. The final mesh has 14,268 elements, 7,319 nodes, and 14,638 in-plane displacement degrees of freedom. MATLAB and Abaqus use the same mesh, material constants, loading, scalar damage model, and Oliver crack-band formula.

The updated results show close agreement in peak response and crack localization. MATLAB predicts a peak load of 3.63982 kN at CMOD 0.022811 mm; Abaqus predicts 3.60913 kN at CMOD 0.022485 mm. The ratio of the peak loads is 1.009, and the ratio of CMOD values at peak is 1.014. Both simulations localize damage upward from the notch, which is the expected mode-I crack path for this geometry (Grégoire et al., 2013). The agreement is important because the Abaqus UMAT independently evaluates the same damage law and Oliver bandwidth inside a commercial finite-element environment (Dassault Systèmes Simulia Corp., 2023; Oliver, 1989).

On the test workstation, MATLAB used one thread and Abaqus used four threads. The MATLAB solver wall-clock time was 547.58 s, while the Abaqus submit-to-completion time was 1996.25 s. MATLAB time was dominated by stiffness assembly, not by the sparse solve. The timing should not be read as a universal speed claim, because solver settings, output requests, hardware, and Abaqus licensing can all change wall-clock time. It does show that a readable MATLAB implementation can still be practical for research-scale 2D crack-band studies.

Table 1: Updated 2D 3PB comparison using the same mesh, material law, and Oliver T3 crack-band bandwidth.

Quantity	MATLAB	Abaqus + UMAT
Peak load (N)	3639.82	3609.13
CMOD at peak (mm)	0.022811	0.022485
Solver/submit wall-clock (s)	547.58	1996.25
End-to-end/internal time (s)	590.54	1968.00

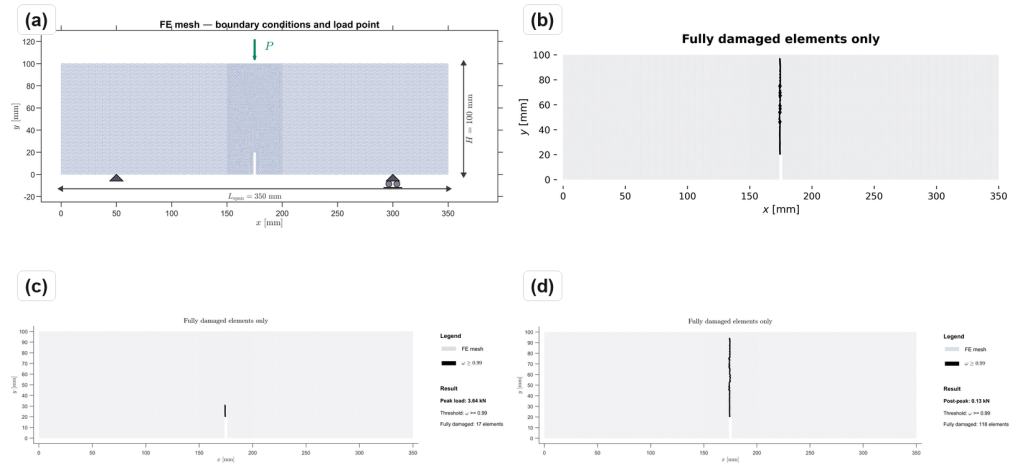


Figure 1: 3PB mesh and damage fields: (a) CPS3 mesh and support/load layout; (b) Abaqus UMAT final fully damaged elements; (c) MATLAB damage field at peak load; (d) MATLAB post-peak damage field.

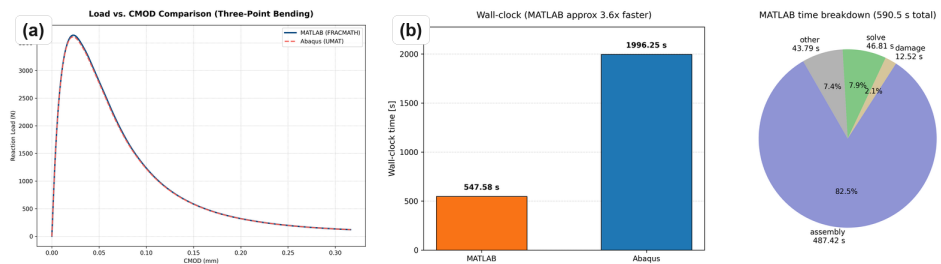


Figure 2: 3PB response and timing: (a) MATLAB and Abaqus load-CMOD response; (b) wall-clock comparison and MATLAB timing breakdown.

108 The Nooru-Mohamed benchmark checks mixed-mode 3D cracking in a 200 mm by 200 mm by
109 50 mm double-edge-notched concrete panel loaded by combined tension and shear (Nooru-
110 Mohamed, 1992). The model follows load path 4a from the original thesis, and the same scalar
111 CDM routine is used with TET4 elements and Oliver crack-band scaling (Nooru-Mohamed,
112 1992; Oliver, 1989). The simulated damage bands initiate at the two notch tips and coalesce
113 across the ligament, matching the experimental crack-path pattern.

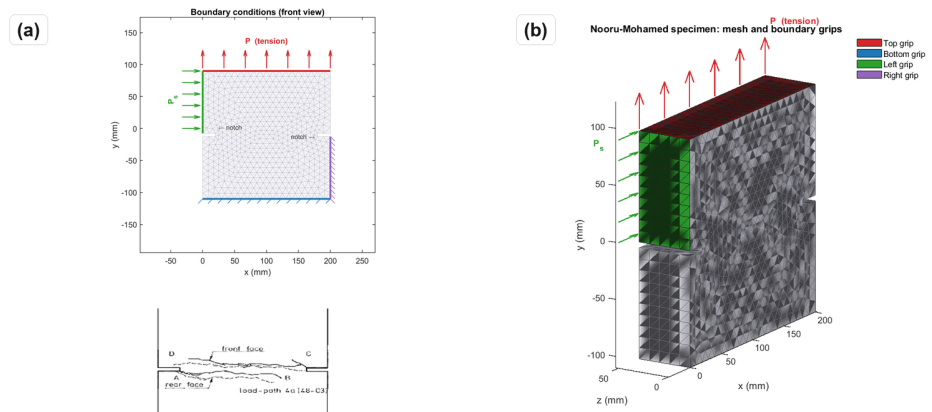


Figure 3: Nooru-Mohamed mixed-mode benchmark: (a) boundary conditions with experimental crack-path inset; (b) 3D mesh.

This example is included because mixed-mode response is a common failure point for simplified fracture implementations. The loading combines a horizontal prescribed displacement, a vertical prescribed displacement, and fixed supports on the opposite edges. In the simulation, the crack-band direction changes as the principal strain field evolves, so the projected bandwidth is recomputed rather than assigned from a constant element size. The resulting band does not remain a straight mode-I notch extension; it bends across the ligament in the same qualitative direction as the reported experimental crack path.

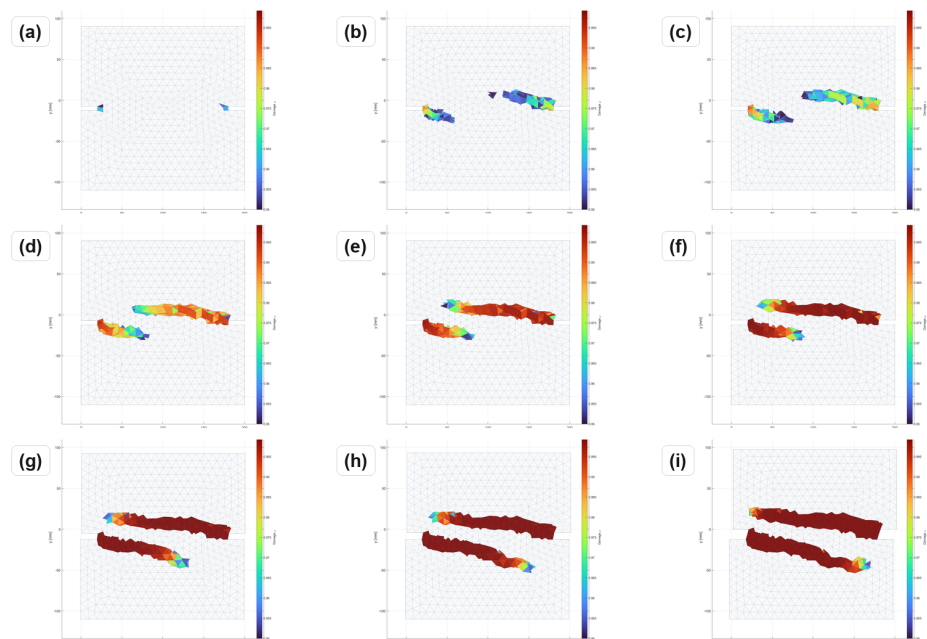


Figure 4: Nooru-Mohamed damage evolution from first localization to coalescence in a 3x3 sequence.

Brokenshire's torsion benchmark tests whether the same formulation can recover a curved 3D fracture surface in a 400 mm by 250 mm by 100 mm plain concrete beam with a 25 mm by 5 mm diagonal notch (Jefferson et al., 2004). The model uses a prescribed twist, TET4 elements, and the same damage update. The computed band nucleates at the notch front and rotates toward the loaded corner, consistent with the experimentally recovered fracture surface.

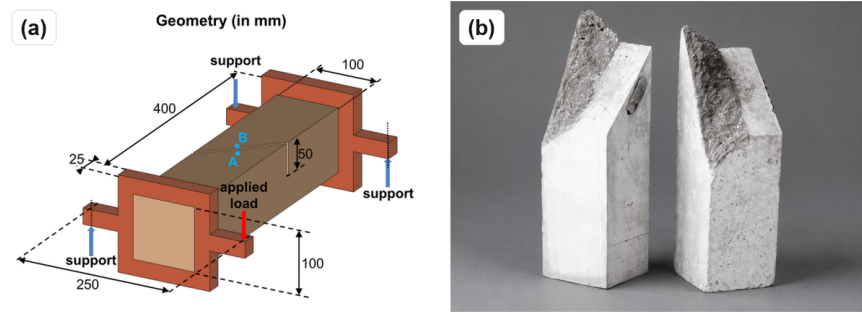


Figure 5: Brokenshire torsion benchmark: geometry and experimental fractured specimen.

The torsion case is deliberately different from the 3PB validation. It contains out-of-plane cracking, a nonuniform stress state, and a visibly curved fracture surface. It therefore checks whether the implementation remains useful beyond a single 2D benchmark. The code stores intermediate damage snapshots, allowing users to inspect how the localized zone grows rather than seeing only the final fully damaged band.

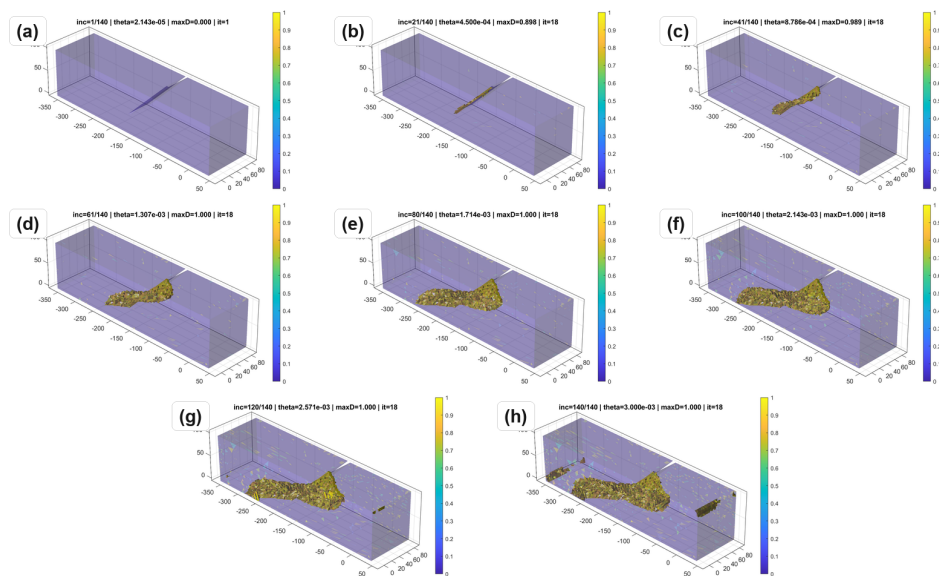


Figure 6: Torsion damage evolution over the imposed twist history.

Software availability

The repository contains MATLAB source code, Abaqus UMAT files, benchmark input decks, plotting scripts, updated results, and a theory manual. MATLAB scripts can be run directly from the benchmark folders, and the plotting scripts regenerate the paper figures from stored CSV files and damage fields. The test suite has been checked on MATLAB R2022a, R2023a, and R2024a across Linux, macOS, and Windows.

Limitations

FRACMATH is quasistatic and does not include inertia, rate effects, contact, plasticity, or unilateral crack closure. Damage is isotropic and scalar, so anisotropic stiffness recovery and cyclic

loading are outside the current scope. Crack-band scaling controls mesh-objective energy dissipation but does not introduce a physical material length scale.

Acknowledgements

The authors acknowledge support from the National Aeronautics and Space Administration, the New Mexico Space Grant Consortium under grant 80NSSC22M0044, and the New Mexico Department of Finance and Administration under grant ZI5044-MG25-109. The opinions and conclusions are those of the authors and do not necessarily reflect the views of the sponsors.

References

- Austrell, P.-E., Dahlblom, O., & others. (2004). *CALFEM — a finite element toolbox, version 3.4*. Lund University.
- Bažant, Z. P., & Oh, B. H. (1983). Crack band theory for fracture of concrete. *Matériaux Et Construction*, 16(3), 155–177. <https://doi.org/10.1007/BF02486267>
- Bažant, Z. P., & Planas, J. (1998). *Fracture and size effect in concrete and other quasibrittle materials*. CRC Press.
- Dassault Systèmes Simulia Corp. (2023). *Abaqus/standard user's manual, version 2023*.
- Davis, T. A. (2004). Algorithm 832: UMFPACK V4.3 — an unsymmetric-pattern multifrontal method. *ACM Transactions on Mathematical Software*, 30(2), 196–199.
- Eldababy, H., Saji, R. P., Pantidis, P., & Mobasher, M. (2025). Parallel-CDM: Parallel implementation of continuum damage mechanics simulations using FEM and MATLAB. *Journal of Open Source Software*, 10(112), 7610. <https://doi.org/10.21105/joss.07610>
- Grégoire, D., Rojas-Solano, L. B., & Pijaudier-Cabot, G. (2013). Failure and size effect for notched and unnotched concrete beams. *International Journal for Numerical and Analytical Methods in Geomechanics*, 37(10), 1434–1452.
- Jefferson, A. D., Bennett, T., & Hee, S. C. (2004). Three-dimensional finite element simulations of fracture tests using the Craft concrete model. *Computers and Concrete*, 1(3), 261–284.
- Logg, A., Mardal, K.-A., & Wells, G. N. (Eds.). (2012). *Automated solution of differential equations by the finite element method: The FEniCS book*. Springer. <https://doi.org/10.1007/978-3-642-23099-8>
- Nooru-Mohamed, M. B. (1992). *Mixed-mode fracture of concrete: An experimental approach* [PhD thesis]. Delft University of Technology.
- Oliver, J. (1989). A consistent characteristic length for smeared cracking models. *International Journal for Numerical Methods in Engineering*, 28(2), 461–474.
- Patzák, B., & Bittnar, Z. (2001). Design of object-oriented finite element code. *Advances in Engineering Software*, 32(10–11), 759–767.
- Richart, N., Anciaux, G., Gallyamov, E., Frérot, L., Kammer, D., Pundir, M., Vocialta, M., Ramos, A. C., Corrado, M., Müller, P., Barras, F., Zhang, S., Ferry, R., Durussel, V., & Molinari, J.-F. (2024). Akantu: An HPC finite-element library for contact and dynamic fracture simulations. *Journal of Open Source Software*, 9(94), 5253. <https://doi.org/10.21105/joss.05253>
- Vree, J. H. P. de, Brekelmans, W. A. M., & Gils, M. A. J. van. (1995). Comparison of nonlocal approaches in continuum damage mechanics. *Computers & Structures*, 55(4), 581–588.